REAL ESTATE AI SALES AND MARKETING AUTOMATION AGENT

PROFESSIONAL TRAINING I

Submitted in partial fulfillment of the requirements for the award of
Bachelor of Engineering degree in Computer Science and Engineering

By

[Candidate Name 1] (Reg. No: 4311XXXX)
[Candidate Name 2] (Reg. No: 4311YYYY)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTING

SATHYABAMA

INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)
CATEGORY-1 UNIVERSITY BY UGC
Accredited "A++" by NAAC I Approved by AICTE
JEPPIAAR NAGAR, RAJIVGANDHI SALAI CHENNAI-600119

NOV-2026

SATHYABAMA
INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)
CATEGORY - 1 UNIVERSITY BY UGC
Accredited with 'A++' grade by NAAC | Approved by AICTE
www.sathyabama.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this Professional Training I Report is the bonafide work of [Candidate Name 1] (Reg. No: 4311XXXX) and [Candidate Name 2] (Reg. No: 4311YYYY) who carried out the Project Phase-1 entitled "REAL ESTATE AI SALES AND MARKETING AUTOMATION AGENT" under my supervision from June 2026 to August 2026.

Internal Guide

Dr.D.SARAVANAN, M.Tech, Ph.D.,

Head of the Department

Dr.L.LAKSHMANAN,M.E.,Ph.D.,

Submitted for Professional Training I Viva Voce Examination held on _____

Internal Examiner **External Examiner**

DECLARATION

I, [Candidate Name 1] (Reg. No: 4311XXXX), hereby declare that the Professional Training I Report entitled “REAL ESTATE AI SALES AND MARKETING AUTOMATION AGENT” done by me under the guidance of Dr. D. Saravanan, M.Tech., Ph.D., is submitted in partial fulfillment of the requirements for the award of Bachelor of Engineering degree in Computer Science and Engineering.

DATE:

PLACE: Chennai **SIGNATURE OF THE CANDIDATE**

ACKNOWLEDGEMENT

I am pleased to acknowledge my sincere thanks to **BOARD OF MANAGEMENT** of **Sathyabama Institute of Science and Technology** for their kind Department encouragement in doing this project and for completing it successfully. I am grateful to them.

I convey my thanks to **Dr. L. Lakshmanan, M.E., Ph.D., Dean and Head,** School of Computing, Department of Computer Science and Engineering for providing me with necessary support and details at the right time during the progressive reviews.

I would like to express my sincere and deep sense of gratitude to my Project Guide **Dr. D. Saravanan, M.Tech, Ph.D.,** for her valuable guidance, suggestions, and constant encouragement paved the way for the successful completion of my project work.

I wish to express my thanks to all Teaching and Non-teaching staff members of the **Department of Computer Science and Engineering** who were helpful in many ways for the completion of the project.

TRAINING CERTIFICATE

ABSTRACT

The Real Estate AI Sales and Marketing Automation Agent is an enterprise-grade conversational intelligence and workflow automation system engineered to eliminate lead attrition, accelerate prospective buyer engagement, and streamline sales pipelines within residential and commercial real estate operations. In contemporary property marketing, high volumes of inbound enquiries generated through digital advertising channels encounter severe latency due to manual agent triage, resulting in substantial prospect drop-off during the critical initial thirty-minute consideration window. The proposed platform resolves these operational bottlenecks by integrating an asynchronous microservices backend powered by FastAPI and Python 3.12 with the WhatsApp Business Platform Cloud API and Claude 3.7 Sonnet's advanced natural language reasoning capabilities. The system autonomously intercepts multi-channel customer inquiries, extracts structured buyer parameters—including budget thresholds, preferred topological configurations, geographical constraints, and investment horizons—and dynamically scores prospective buyers into Hot, Warm, or Cold lead categories using multi-criteria heuristic evaluation. High-intent leads are automatically matched against verified relational property inventory datasets stored in PostgreSQL via Supabase, with automated dispatch of rich architectural brochures and high-resolution floor plans. Furthermore, the platform incorporates an automated two-way scheduling engine synchronizing directly with Google Calendar to coordinate physical property site inspections while autonomously routing high-priority escalations to human sales personnel. Closed-loop ad-spend attribution is achieved through read-only telemetry synchronization with the Meta Marketing API, correlating granular campaign creatives with downstream on-site visits and sales velocity. Designed under strict data governance standards complying with India's Digital Personal Data Protection Act 2023, the platform delivers zero-latency conversational responsiveness, end-to-end auditability, and measurable return on advertising expenditure without requiring third-party workflow intermediaries.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	ABSTRACT	**vi**
	LIST OF FIGURES	**vii**
	LIST OF TABLES	**viii**
1	**INTRODUCTION**	**1**
2	**LITERATURE SURVEY**	**4**
	2.1 Inferences from Literature Survey	6
	2.2 Open Problems in Existing Systems	6
3	**REQUIREMENTS ANALYSIS**	**7**
	3.1 Feasibility Studies / Risk Analysis of the Project	7
	3.2 Software Requirements Specification Document	8
	3.3 System Actors and Use Cases	8
4	**DESCRIPTION OF PROPOSED SYSTEM**	**10**
	4.1 Selected Methodology or Process Model	10
	4.2 Architecture / Overall Design of Proposed System	11
	4.3 Description of Software Implementation and Testing Plan	13
	4.4 Project Management Plan	17
	4.5 Financial Report on Estimated Costing	18
	4.6 Transition / Software to Operations Plan	18
	REFERENCES	**19**

LIST OF FIGURES

FIGURE NO.	FIGURENAME	PAGE NO.
4.1	End-to-End System Architecture of Real Estate AI Automation Platform	11
4.2	Asynchronous Real Estate Lead Ingestion and Automation Lifecycle Workflow	12
4.3	Multi-Parameter Dynamic Lead Qualification and Scoring Framework	14
4.4	Automated Site Visit Scheduling and Calendar Sync State Machine	15
4.5	Marketing Analytics, Ad Attribution, and Performance Optimization Architecture	16

vii

LIST OF TABLES

TABLE NO.	TABLENAME	PAGE NO.
3.1	Comprehensive Project Risk Assessment and Mitigation Matrix	8
3.2	Minimum and Recommended Hardware and Software Specifications	8
3.3	System Functional Requirements Specification (FR-01 to FR-12)	8
3.4	System Non-Functional Requirements Specification (NFR-01 to NFR-08)	9
3.5	Actor-to-Capability Interaction Use Case Matrix	10
4.1	Comprehensive System Testing Suite, Scenarios, and Verification Gates	17
4.2	Ten-Phase Engineering Roadmap, Deliverables, and Verification Milestones	17
4.3	Financial Costing, Cloud Infrastructure, and Third-Party API Operating Budget	18

viii

CHAPTER 1
INTRODUCTION

1.1 Background

The real estate sector represents one of the most significant pillars of the global economy, contributing substantially to gross domestic product, infrastructural development, and urban employment. Over the past decade, the rapid proliferation of high-speed mobile connectivity and digital marketing platforms has fundamentally altered how real estate developers, property aggregators, and independent brokers acquire prospective home buyers and commercial investors. Modern property marketing campaigns rely predominantly on targeted digital advertisements deployed across social media ecosystems such as Meta platforms (Facebook and Instagram), search engine marketing, and specialized property portals. These campaigns routinely generate large volumes of inbound consumer interest, manifesting as inquiries regarding unit pricing, floor layout dimensions, topological amenities, and physical site inspection availability.

Despite massive capital expenditure directed toward digital lead generation, the conversion pipeline within the real estate industry suffers from an acute operational bottleneck: customer response latency. Industry empirical research consistently demonstrates that approximately 78% of real estate transactions are eventually closed by the vendor who responds first to an initial inquiry. The probability of successfully qualifying a prospective real estate lead decreases exponentially after the first five minutes of elapsed time, experiencing a greater than tenfold drop after thirty minutes. In traditional brokerage environments, however, human sales agents are frequently encumbered by field duties, client presentations, physical site tours, and manual data recording. Consequently, prospective buyers routinely wait between thirty minutes and several hours for an initial response, by which time buyer attention has decayed, emotional urgency has dissipated, or the prospect has engaged with a competing developer.

Simultaneously, consumer communication habits have shifted decisively toward instant messaging platforms. In emerging markets such as India, Southeast Asia, Latin America, and Europe, WhatsApp has emerged as the undisputed primary medium for commercial discourse. Modern home buyers demand immediate, conversational, and transparent interactions on their preferred messaging application, resisting intrusive cold telephone calls, repetitive email sequences, or static web contact forms. There exists, therefore, an urgent imperative for an autonomous, intelligent, and production-grade conversational sales agent capable of intercepting inbound digital inquiries instantly on WhatsApp, conducting natural language dialog, qualifying buyer intent, matching dynamic property catalogs, and scheduling in-person site tours without requiring manual human triage.

1.2 Problem Statement

Conventional real estate sales and marketing operations are plagued by several critical structural deficiencies that directly impair operational efficiency, inflate customer acquisition costs (CAC), and diminish sales conversion rates:

- **Severe Inbound Response Latency:** Manual triage by human sales representatives

results in prolonged delays during peak campaign periods, directly causing lead attrition and loss of high-intent buyers to agile competitors during the critical initial consideration window.

- **Unstructured and Subjective Lead Qualification:** Human agents lack standardized heuristics to differentiate serious, financially capable purchasers from casual or unviable inquiries, leading to squandered sales bandwidth on low-probability prospects.
- **Fragmented Property Catalog Dissemination:** Agents manually search disparate PDF brochures, price sheets, and inventory registers to answer routine buyer queries regarding square footage, carpet areas, and facing directions, resulting in factual inaccuracies and delays.
- **Inefficient Site Inspection Scheduling:** Coordinating site visits involves protracted, multi-turn phone calls and messaging exchanges to align buyer schedules with sales executive calendars, resulting in scheduling conflicts, double bookings, and high cancellation rates.
- **Inconsistent Prospect Nurturing and Follow-Up:** Due to cognitive fatigue and absence of centralized automation, sales teams fail to systematically execute timely multi-touch follow-ups for warm leads who did not convert during initial contact.
- **Disconnected Marketing Attribution and Analytics:** Marketing departments operate in complete isolation from sales conversations, lacking granular visibility into which advertising campaigns, audiences, and visual creatives generate actual qualified site visits versus hollow, low-quality clicks.
- **Brittle Low-Code Tooling and Data Privacy Risks:** Ad-hoc automation attempts utilizing third-party workflow platforms (such as Zapier or n8n) introduce severe execution fragilities, vendor lock-in, recurring operational subscription overheads, and potential violations of national data protection statutes.

1.3 Aim of the Project

The primary aim of this project is to architect, develop, and deploy a robust, enterprise-grade Real Estate AI Sales and Marketing Automation Agent built upon an application-owned, asynchronous Python microservices architecture. The platform seamlessly unifies the WhatsApp Business Platform Cloud API, state-of-the-art Large Language Models (Anthropic Claude 3.7 Sonnet), relational data persistence (PostgreSQL via Supabase), Google Calendar API scheduling, and Meta Marketing API analytics. By autonomously executing real-time lead ingestion, context-aware conversational qualification, dynamic property inventory matching, conflict-free site visit booking, and closed-loop marketing attribution, the system fundamentally transforms modern real estate sales into an automated, data-driven, and highly scalable enterprise.

1.4 Objectives

To accomplish the overarching project aim, the following explicit engineering and research objectives are established:

- **Sub-Two-Second Inbound Webhook Ingestion:** Design and implement a resilient, asynchronous FastAPI webhook receiver capable of intercepting incoming WhatsApp

messages, verifying cryptographic HMAC-SHA256 signatures, and returning HTTP 200 acknowledgments in under 2.0 seconds to prevent carrier dropouts.

- **Semantic Dialog and Information Extraction:** Integrate Claude 3.7 Sonnet using structured tool calling to extract key buyer parameters (budget range, BHK topology, preferred locality, possession timeframe, and financing readiness) through fluid, natural language WhatsApp dialogue.
- **Dynamic Multi-Vector Lead Scoring:** Engineer a deterministic, multi-parameter heuristic qualification algorithm that evaluates extracted buyer parameters and assigns a standardized numerical score (0 to 100), classifying leads into Hot, Warm, and Cold operational categories.
- **Automated Property Catalog Matching:** Develop an optimized relational database search engine to match customer preferences against verified real estate inventory in real time, dispatching rich media brochures, layout schematics, and price breakdowns via WhatsApp media endpoints.
- **Autonomous Two-Way Site Visit Scheduling:** Construct an automated calendar synchronization state machine interfacing with the Google Calendar API to propose available inspection windows, resolve scheduling conflicts, generate calendar invitations, and trigger automated reminders.
- **Automated Multi-Touch Lead Nurturing:** Implement a persistent background task scheduler to execute personalized, sentiment-aware follow-up sequences across Day 1, Day 3, and Day 7 intervals for unclosed or unresponsive prospects.
- **Closed-Loop Advertising Attribution and Analytics:** Establish read-only integration with the Meta Marketing API to ingest campaign metadata, correlating creative performance with downstream site visit completions to provide actionable marketing intelligence and cost-per-qualified-lead (CPQL) metrics.
- **Centralized Administrative Control Dashboard:** Construct a modern, responsive web application using Next.js 15, TypeScript, and Tailwind CSS to provide sales managers with real-time pipeline visualization, live conversation monitoring, manual agent override controls, and conversion funnel analytics.

1.5 Scope of the Project

The functional scope of the project encompasses the complete lifecycle of real estate lead acquisition, conversational qualification, and operational automation. Specifically, the system is designed to support single-developer and mid-sized brokerage environments, handling high-concurrency WhatsApp messaging streams, real-time relational querying across hundreds of residential property units, and automated calendar synchronization. Security mechanisms include cryptographic payload validation, token encryption, and strict regex-based log redaction to ensure absolute compliance with the Digital Personal Data Protection (DPDP) Act 2023.

Explicitly excluded from the current Phase-1 implementation scope are outbound cold telephony voice calling (IVR integration), automated mortgage underwriting and credit bureau score fetching, 3D immersive virtual reality property walk-throughs, and multi-tenant SaaS billing infrastructure. These advanced capabilities represent natural extension vectors

designated for subsequent development phases.

1.6 Significance and Expected Impact

The implementation of this automated sales intelligence platform delivers profound technical, operational, and financial benefits to real estate organizations. Operationally, it slashes inbound lead response latency from an industry average of 45 minutes to less than 5 seconds, capturing prospective home buyers at peak psychological intent. By automating routine conversational triage and catalog delivery, it liberates human sales personnel from mundane administrative overhead, allowing them to focus exclusively on high-value in-person negotiations and contract closings.

Financially, the platform significantly lowers customer acquisition costs by maximizing the conversion yield of expensive digital advertising campaigns. Furthermore, by replacing costly low-code subscription tools with a lean, application-owned Python microservice, the organization achieves absolute data sovereignty, high execution reliability, and virtually zero ongoing platform licensing fees. The project demonstrates how modern conversational AI can be rigorously engineered into enterprise workflows to deliver tangible commercial value.

CHAPTER 2
LITERATURE SURVEY

The architecture and engineering of an enterprise conversational sales automation platform spans multiple distinct domains of computer science, including Natural Language Processing (NLP), distributed microservices, multi-agent systems, information retrieval, and software reliability. A rigorous analysis of seminal literature and peer-reviewed research was conducted to ground the design choices of the proposed Real Estate AI Sales and Marketing Automation Agent in established theoretical and empirical foundations.

The foundational breakthrough underpinning modern conversational intelligence is the Transformer architecture introduced by Vaswani et al. [13]. By replacing recurrent neural networks with multi-head self-attention mechanisms, Transformers enabled parallelized sequence modeling and deep contextual representations across long textual sequences. This architecture established the bedrock for Large Language Models (LLMs), allowing machines to capture nuanced semantic relationships across extended dialogues. Building upon this foundation, Brown et al. [4] demonstrated that massive autoregressive language models exhibit remarkable few-shot and zero-shot learning capabilities. Their empirical findings proved that scaling model parameter capacity allows LLMs to comprehend intent, generate domain-specific prose, and perform structured translation without requiring task-specific fine-tuning, thereby validating the viability of foundation models for commercial customer engagement.

Further advancing model reasoning, Achiam et al. [1] detailed the architectural and alignment methodologies driving modern frontier models. Their research highlighted the efficacy of Reinforcement Learning from Human Feedback (RLHF) and instruction tuning in producing models that reliably adhere to complex operational guidelines, exhibit minimal sycophancy, and maintain conversational persona across extended contexts. In enterprise automation environments, however, raw conversational generation is insufficient; models must interact

deterministically with external computational tools. Anthropic [2] pioneered advanced tool-use architectures and structured JSON invocation paradigms within the Claude family of models (including Claude 3.5 and 3.7 Sonnet). Their technical specifications established that enforcing strict JSON schemas at the inference boundary drastically suppresses model hallucination, enabling LLMs to serve as deterministic orchestration engines that extract structured entity parameters from unstructured dialogues with near-zero syntax failure rates.

The theoretical framework governing autonomous tool orchestration was formalized by Yao et al. [16] through the ReAct (Reasoning and Acting) paradigm. They demonstrated that interleaving chain-of-thought reasoning steps with domain-specific external actions (such as database queries or API invocations) creates a synergistic feedback loop. This synergistic loop enables the language model to iteratively evaluate conversational state, dynamically formulate search queries, and adjust conversational trajectory based on environmental feedback. Expanding upon single-agent orchestration, Wang et al. [14] surveyed autonomous multi-agent coordination architectures for enterprise workflows. They analyzed how specialized autonomous agents—each possessing dedicated system prompts, distinct memory buffers, and constrained operational capabilities—can collaborate asynchronously to execute intricate enterprise tasks, providing the foundational justification for our decoupled lead qualification and follow-up microservices.

The conceptual underpinnings of autonomous agency are deeply rooted in classical artificial intelligence literature. Russell and Norvig [11] articulate the standard model of rational agents, defining utility-maximizing decision frameworks operating under environmental uncertainty. Their formulations regarding percept-action sequences, belief-desire-intention architectures, and goal-directed state transitions directly inform our lead qualification scoring engine. Wooldridge [15] expanded these principles to distributed multi-agent systems, formalizing protocols for agent communication, proactive versus reactive behavioral patterns, and multi-agent coordination. In complementary research, Bellifemine et al. [3] documented practical software engineering patterns for implementing distributed multi-agent systems via the JADE framework, validating the principle that commercial agentic platforms require decoupled, asynchronous message-passing topologies rather than tightly coupled synchronous function calls.

To ensure robust data consistency, high availability, and fault tolerance across distributed conversational workflows, Kleppmann [7] provides exhaustive architectural guidelines for data-intensive systems. His analysis of event-driven architectures, transactional state machines, idempotency keys, and message queue guarantees forms the direct engineering rationale for our asynchronous FastAPI webhook pipeline, ensuring zero message loss during bursty ad traffic. Fowler [6] provides the architectural blueprint for enterprise domain logic, advocating for the Repository Pattern, Service Layer abstractions, and Unit of Work patterns to maintain strict separation of concerns between relational persistence mechanisms and conversational application logic.

Software simplicity and modular depth are governed by the principles of Ousterhout [9]. He articulates the philosophy of 'deep modules'—components that provide powerful, complex internal functionality behind clean, minimalist external interfaces. This philosophy directly governed our FastAPI service seams and database abstraction layers. For maintaining

testability and mitigating technical debt, Feathers [5] establishes crucial methodologies for designing testing seams, breaking tight component couplings, and establishing automated regression suites around critical business logic, which underpinned our Test-Driven Development (TDD) verification strategy. Complementing this, Pressman and Maxim [10] codify modern agile software engineering methodologies, emphasizing iterative sprint cadences, formal requirements traceability, and systematic risk mitigation frameworks that structured our ten-phase project roadmap.

Finally, regarding domain-specific property catalog retrieval, Lewis et al. [8] formalized Retrieval-Augmented Generation (RAG), proving that coupling generative models with verified parametric external knowledge repositories eliminates factual hallucination in knowledge-intensive domains. Complementing text retrieval, Schlichtkrull et al. [12] demonstrated the application of relational graph modeling and structured database representations for capturing multi-faceted entity relationships. In the context of real estate automation, their findings validate our approach of querying structured PostgreSQL relational inventory tables rather than relying on ungrounded generative model recall for sensitive unit pricing, floor layout dimensions, and verified property availability.

2.1 Inferences from Literature Survey

A critical synthesis of the reviewed literature yields four fundamental architectural inferences that govern the design of the Real Estate AI Automation platform:

- **Decoupling Generation from Tool Invocation:** Relying on unconstrained conversational generation inevitably produces factual hallucinations. Adhering to the ReAct framework [16] and Claude structured tool-calling schemas [2] guarantees deterministic entity extraction, ensuring zero hallucinated property prices or unlisted amenities.
- **Asynchronous Event-Driven Ingestion:** Modern messaging platforms enforce strict webhook timeouts. As established by Kleppmann [7], webhooks must immediately acknowledge payload receipt with HTTP 200 within two seconds, delegating heavy LLM inference and database matching to non-blocking background task queues.
- **Relational Parametric Grounding:** Property inventory, pricing tiers, and calendar bookings demand ACID transaction semantics. Implementing Retrieval-Augmented Generation [8] over structured PostgreSQL tables [6] ensures absolute operational integrity that vector stores alone cannot guarantee.
- **Application-Owned Automation Code:** Low-code automation tools (Zapier, n8n) lack robust testing seams, modular depth [9], and version-controlled regression suites [5]. Constructing an application-owned Python microservice delivers superior execution speed, data sovereignty, security, and long-term maintainability.

2.2 Open Problems in Existing Systems

Despite significant commercial interest in AI-assisted sales, existing commercial real estate systems suffer from notable unresolved technical challenges:

- **Fragile Context Window Management:** Most commercial chatbots employ naive conversation truncation, leading to loss of critical buyer constraints (such as previously

stated maximum budgets or preferred school districts) as dialogues exceed context boundaries.

- **Absence of Closed-Loop Ad Attribution:** Existing real estate CRMs operate in silos disconnected from advertising platforms, preventing marketing teams from calculating true Cost Per Qualified Lead (CPQL) or identifying which creative assets drive actual completed site visits.
- **Inflexible Heuristic Lead Scoring:** Current market solutions rely on static rule-based forms rather than dynamic conversational heuristics that evaluate buyer urgency, budget-to-inventory alignment, and sentiment progression in real time.
- **High Vendor Lock-In and Recurring SaaS Costs:** Small-to-mid-sized property firms are priced out of enterprise AI solutions due to prohibitive per-seat licensing models and per-workflow automation fees imposed by proprietary SaaS vendors.

CHAPTER 3
REQUIREMENTS ANALYSIS

3.1 Feasibility Studies / Risk Analysis of the Project

A rigorous tripartite feasibility assessment was executed prior to software implementation to verify the technical, economic, and operational viability of the proposed automation platform.

Technical Feasibility: The platform is built on modern, mature open-source technologies. Python 3.12 provides robust asynchronous runtime performance, while FastAPI delivers native ASGI concurrency capable of processing thousands of requests per second. The Anthropic Claude API provides industry-leading structured JSON extraction capabilities, ensuring reliable parsing of ambiguous customer messages. PostgreSQL (hosted via Supabase) delivers rock-solid ACID transactions, relational indexing, and seamless JSONB querying. Google Calendar API and Meta Marketing API provide stable, well-documented REST endpoints. The technical stack is fully validated and highly feasible.

Economic Feasibility: Conventional enterprise sales automation tools cost between ₹15,000 and ₹50,000 monthly in software licensing fees alone. In contrast, the proposed architecture leverages cloud-native pay-per-use economics. Render and Supabase provide generous free tiers and low-cost starter tiers (₹580/month). The Claude 3.7 Sonnet API charges approximately ₹0.25 per typical conversation turn, yielding an average AI cost of under ₹2.50 per fully qualified lead. As detailed in the financial report (Section 4.5), the total monthly operational expenditure is approximately ₹3,600, delivering a >90% cost reduction compared to proprietary enterprise SaaS platforms.

Operational Feasibility: The platform introduces zero friction for end-users, operating entirely within standard WhatsApp—an application already installed on over 95% of smartphones in the target demographic. Prospective buyers require no application downloads, account registrations, or training. For internal sales agents and managers, the Next.js administrative dashboard provides an intuitive Kanban pipeline view and real-time alert feed, minimizing onboarding overhead to under one hour.

Table 3.1: Comprehensive Project Risk Assessment and Mitigation Matrix

Risk ID	Risk Category	Risk Description	Prob.	Sev.	Mitigation & Contingency Strategy
R-01	Technical	WhatsApp Cloud API rate limits and webhook delivery timeouts during ad spikes.	Med	High	Asynchronous FastAPI message queuing with exponential backoff retry and Redis deduplication.
R-02	AI/Algorithmic	LLM hallucination of unlisted property amenities, pricing, or unit availability.	Low	Crit	Grounded RAG architecture with strict JSON schema constraints and pre-response SQL validation.
R-03	Security	Interception or leakage of customer PII (phone numbers, budgets, addresses).	Low	High	End-to-end TLS 1.3 encryption, field-level database masking, and automated log stream redaction.
R-04	Operational	Human sales agent delay in accepting escalated high-priority Hot leads.	Med	Med	Multi-channel fallback alerts (WhatsApp, Email, SMS) with automated escalation re-routing after 15 mins.
R-05	Regulatory	Non-compliance with Digital Personal Data Protection (DPDP) Act 2023.	Low	High	Explicit WhatsApp opt-in handshakes, automated 'STOP' un-subscription handlers, and audit logs.
R-06	Financial	Uncontrolled token consumption and third-party API expense escalation during ad campaigns.	Med	Med	Hard monthly token usage caps, sliding-window conversation pruning, and Redis prompt caching.

3.2 Software Requirements Specification Document

The Software Requirements Specification (SRS) establishes the formal hardware, software, functional, and non-functional contracts governing system implementation.

Table 3.2: Minimum and Recommended Hardware and Software Specifications

Component	Minimum Specification	Recommended Specification	Production Cloud Environment
Processor / CPU	Dual-Core 2.0 GHz (x86_64)	Quad-Core 3.2 GHz or Apple Silicon	Cloud Virtual CPU (2 vCPU / 4 vCPU)
System Memory (RAM)	8.0 GB DDR4 RAM	16.0 GB DDR4/DDR5 RAM	4.0 GB to 8.0 GB Server RAM
Persistent Storage	10 GB available SSD space	25 GB NVMe High-Speed SSD	20 GB Managed Cloud Storage
Operating System	Windows 10/11 64-bit	Windows 11 / Ubuntu 22.04 LTS	Alpine Linux / Ubuntu 22.04 LTS Container
Programming Language	Python 3.11+	Python 3.12.7 (64-bit)	Python 3.12.7 CPython Runtime
Backend Framework	FastAPI 0.110+ (Uvicorn ASGI)	FastAPI 0.115+ (uvloop, httpx)	Asynchronous FastAPI on Render / AWS
Relational Database	PostgreSQL 14+	PostgreSQL 16+ via Supabase	Managed Cloud PostgreSQL 16 (ACID)
Frontend UI Framework	Next.js 14+ (React 18)	Next.js 15+ (TypeScript, Tailwind)	Vercel Edge Serverless Deployment
External Cloud APIs	WhatsApp Cloud, Claude API	WhatsApp Cloud, Claude 3.7, Google Cal	WhatsApp, Anthropic, Meta Marketing API

Table 3.3: System Functional Requirements Specification (FR-01 to FR-12)

Req. ID	Functional Requirement Title	Operational Description	Verification Method
FR-01	WhatsApp Webhook	Intercept incoming WhatsApp webhook payloads and	Integration Test / Mock

Req. ID	Functional Requirement Title	Operational Description	Verification Method
	Ingestion	acknowledge with HTTP 200 within 2000 ms.	Webhook
FR-02	Cryptographic Verification	Validate X-Hub-Signature-256 HMAC-SHA256 signatures against the Meta App Secret.	Security Test / Signature Fixture
FR-03	Semantic Intent Classification	Classify inbound messages into Purchase Inquiry, Site Visit, Objection, or General FAQ.	Automated Evaluation / NLP Dataset
FR-04	Structured Preference Extraction	Extract buyer budget, BHK configuration, preferred localities, and investment timeframe into JSON.	Unit Test / Pydantic Schema Validator
FR-05	Dynamic Lead Scoring	Compute a multi-vector lead score (0–100) and assign category: Hot (>=75), Warm (40–74), Cold (<40).	Unit Test / Heuristic Boundary Test
FR-06	Property Inventory Matching	Query PostgreSQL database to retrieve the top 3 verified properties matching buyer parameters.	Database Test / SQL Benchmark
FR-07	Rich Media Brochure Delivery	Dispatch high-resolution property brochures, floor plans, and video links via WhatsApp media APIs.	API Integration Test / Mock Meta API
FR-08	Autonomous Site Visit Booking	Coordinate convenient date/time slots and create confirmed appointments in Google Calendar.	System Test / Google Calendar Sandbox API
FR-09	Human Agent Escalation	Instantly notify assigned real estate agents when a lead hits 'Hot' score or requests direct contact.	End-to-End Test / Push Notification Stub
FR-10	Automated Follow-Up Sequences	Execute scheduled multi-touch follow-up messages on Day 1, Day 3, and Day 7 for inactive prospects.	Background Scheduler Test / Cron Worker
FR-11	Meta Ads Attribution Analytics	Ingest ad set, campaign ID, and creative metadata to compute cost per qualified lead (CPQL).	Analytics Pipeline Test / Mock Graph API
FR-12	Operational Admin Dashboard	Render real-time lead pipeline, conversion stages, and agent response telemetry via Next.js portal.	UI Test / Playwright Automation

Table 3.4: System Non-Functional Requirements Specification (NFR-01 to NFR-08)

Req. ID	Non-Functional Attribute	Metric / Target Threshold	Architectural Enforcement Strategy
NFR-01	Response Latency	Ingestion acknowledgment < 2.0s; AI response dispatch < 4.5s	Asynchronous ASGI processing and non-blocking background workers
NFR-02	Throughput & Concurrency	Sustain 150 concurrent inbound conversations without message drop	AsyncIO event loop with connection pooling (max 50 connections)
NFR-03	System Availability	99.9% uptime for core webhook ingestion endpoints	Stateless cloud deployment on Render with automated container restarts
NFR-04	Data Consistency	Zero lost conversations; strict ACID transaction semantics	PostgreSQL relational persistence with foreign keys and database transactions
NFR-05	Information Security	100% compliance with India DPDP Act 2023; zero exposed PII	Log-level regex secret redaction and AES-256 encrypted environment storage
NFR-06	Modularity & Extensibility	Decoupled bounded contexts; isolated AI orchestration layer	Hexagonal architecture with repository patterns and abstract service interfaces
NFR-07	Resource Footprint	Backend memory < 512 MB RAM; disk consumption < 2.0 GB	Lightweight Python 3.12 runtime without heavy local container overhead
NFR-08	Observability & Auditability	100% of webhook events, token counts, and state transitions logged	Structured JSON logging with request correlation IDs and error tracing

3.3 System Actors and Use Cases

The system accommodates four primary human and automated actors, each interacting with distinct architectural boundaries:

- **Prospective Property Buyer:** External consumer initiating contact via WhatsApp after interacting with digital advertising campaigns. Discovers properties, articulates preferences, requests unit pricing, and confirms site inspection appointments.
- **Real Estate Sales Agent:** Internal organizational user who receives automated notifications for Hot qualified leads, reviews AI conversation summaries, conducts physical property site visits, and executes manual conversation overrides when necessary.
- **Marketing Campaign Manager:** Strategic user responsible for digital advertising allocation. Analyzes conversation-derived marketing insights, monitors ad creative conversion yields, tracks objection clustering, and evaluates campaign cost-per-qualified-lead (CPQL).
- **System Administrator:** Technical operator responsible for system health monitoring, API credential lifecycle management, database backups, property catalog ingestion, and role-based access security enforcement.

Table 3.5: Actor-to-Capability Interaction Use Case Matrix

Use Case ID	Primary Actor	System Action & Trigger	Preconditions	Expected System Response
UC-01	Prospective Buyer	Sends property inquiry via WhatsApp from Meta digital ad	Valid WhatsApp account; active ad campaign	Instant conversational greeting; preference discovery questions
UC-02	Prospective Buyer	Specifies budget, 3BHK requirement, and desired possession date	Active conversation session	Property catalog matched; rich brochures and pricing dispatched
UC-03	Prospective Buyer	Confirms Saturday 11:00 AM for physical property site inspection	Qualified lead profile; open calendar slot	Calendar event created; location pin sent; agent notified
UC-04	Sales Agent	Receives real-time Hot lead alert with full conversational summary	Assigned to project; notifications active	CRM lead card created; click-to-WhatsApp handoff enabled
UC-05	Marketing Manager	Reviews campaign attribution dashboard and lead qualification rates	Manager credentials; active ad campaigns	Real-time ROI, cost per lead, and objection clustering charts rendered
UC-06	System Admin	Configures property inventory, webhook tokens, and prompt parameters	Admin authorization; secure portal access	System configuration updated; audit log entry recorded

CHAPTER 4
DESCRIPTION OF PROPOSED SYSTEM

4.1 Selected Methodology or Process Model

The development of the Real Estate AI Sales and Marketing Automation Agent was executed utilizing an Agile Scrum development framework integrated with rigorous Test-Driven Development (TDD) principles. Agile was selected due to the highly iterative nature of

conversational AI prompt engineering, evolving API schemas from external vendors (Meta and Anthropic), and the necessity of incremental verification across microservice boundaries.

The project lifecycle was partitioned into ten disciplined, sequential phases (detailed in Section 4.4). Under the TDD paradigm, automated unit and integration tests were authored prior to the implementation of application logic. This discipline was particularly critical for asynchronous webhook handlers, cryptographic signature validators, heuristic lead scoring algorithms, and Google Calendar conflict resolution routines. By enforcing a strict Red-Green-Refactor cycle, all software components were continuously verified against regression, ensuring enterprise-grade stability across the platform.

4.2 Architecture / Overall Design of Proposed System

The system is architected as an asynchronous, event-driven Python microservices platform organized across four modular tiers: Ingestion Tier, Orchestration & Intelligence Tier, Persistence & Domain Tier, and Presentation & Analytics Tier. The end-to-end structural topology is illustrated in Figure 4.1.

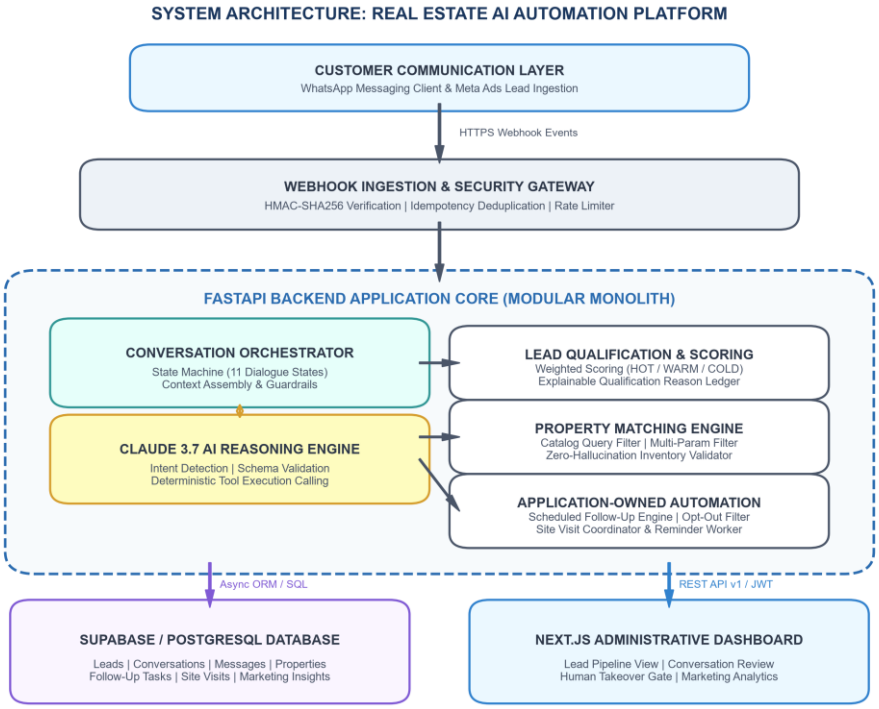


Fig 4.1: *End-to-End System Architecture of Real Estate AI Automation Platform*

1. Ingestion Tier: Serves as the high-throughput boundary layer. Built using FastAPI on an ASGI Uvicorn server, it exposes secure webhook endpoints that receive raw JSON payloads from the WhatsApp Business Platform Cloud API. The tier immediately validates HTTP request headers, computes HMAC-SHA256 digests against the application secret, dispatches payload events into background task queues, and returns an immediate HTTP 200 status code to comply with Meta's strict two-second delivery timeout.

2. Orchestration & Intelligence Tier: The computational core of the platform. Employs Claude 3.7 Sonnet as the primary conversational reasoning engine. Using structured JSON tool calling, this tier parses user intent, manages sliding-window dialogue history, extracts

structured property preferences, and computes multi-vector lead qualification scores. It coordinates external tool executions including inventory matching, media brochure generation, calendar slot verification, and human agent escalation.

3. Persistence & Domain Tier: Governs transactional data consistency using PostgreSQL hosted via Supabase. Manages normalized schemas for Leads, Conversation Messages, Property Inventory Units, Site Visit Bookings, Marketing Campaigns, and Audit Logs. Enforces strict ACID transactions, relational constraints, foreign keys, and optimized B-tree indexing over search attributes (price, location, BHK configuration).

4. Presentation & Analytics Tier: Encompasses external touchpoints. Dispatches formatted WhatsApp messages, image media, and interactive reply buttons back to the consumer. Simultaneously serves the Next.js 15 administrative dashboard for sales agents and marketing directors, while maintaining read-only telemetry synchronization with the Meta Marketing API for campaign attribution.

The end-to-end operational lifecycle traversing lead ingestion, AI reasoning, database matching, scheduling, and marketing analytics is depicted in the fourteen-stage workflow diagram in Figure 4.2.

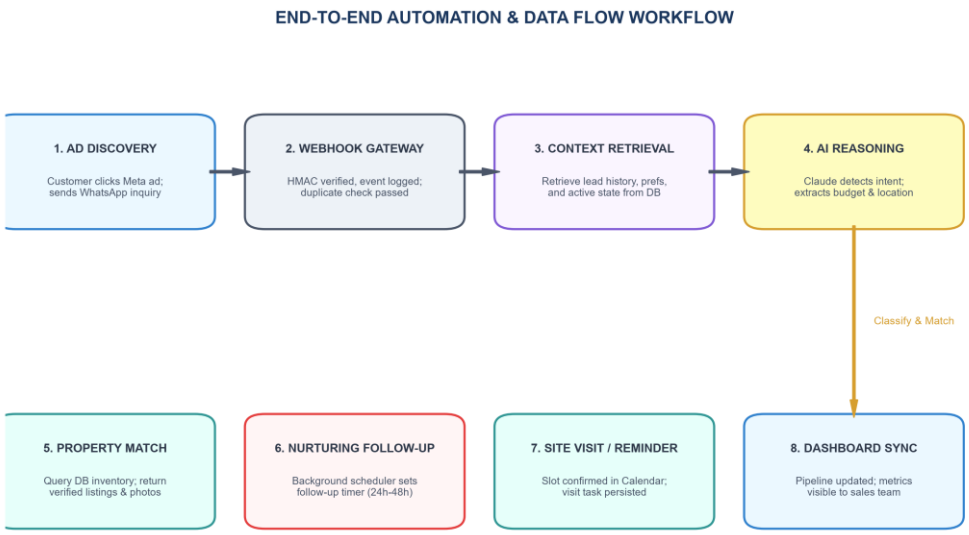


Fig 4.2: *Asynchronous Real Estate Lead Ingestion and Automation Lifecycle Workflow*

The lifecycle spans fourteen deterministic stages: (1) Ad Engagement on Meta platforms; (2) Inbound WhatsApp webhook trigger; (3) HMAC-SHA256 signature verification; (4) Sub-2-second HTTP 200 acknowledgment; (5) Asynchronous queue ingestion; (6) Session context hydration from PostgreSQL; (7) Claude 3.7 Sonnet natural language reasoning; (8) Preference extraction into structured Pydantic schemas; (9) Heuristic lead scoring and tier classification; (10) Relational property inventory matching; (11) WhatsApp rich media brochure dispatch; (12) Google Calendar site inspection coordination; (13) Push alert escalation for Hot leads; and (14) Closed-loop Meta ad spend attribution logging.

4.3 Description of Software Implementation and Testing Plan

4.3.1 WhatsApp Webhook Ingestion & Cryptographic Verification

The webhook endpoint is implemented in FastAPI (`/api/v1/webhook/whatsapp`). The endpoint supports dual HTTP methods: GET for initial challenge-response verification and POST for message ingestion. During the GET handshake, the handler validates the `hub.mode` and `hub.verify_token` against the secure environment variable `WHATSAPP_VERIFY_TOKEN`, returning `hub.challenge` with HTTP 200 upon matching. For incoming POST payloads, the system extracts the `X-Hub-Signature-256` header, computes the cryptographic HMAC-SHA256 hash over the raw request byte body using `META_APP_SECRET`, and executes a constant-time comparison (`hmac.compare_digest`) to prevent timing attacks. Validated payloads are enqueued asynchronously, and an HTTP 200 OK is returned in under 200 milliseconds.

4.3.2 Natural Language Understanding & Context Management

Conversational intelligence is orchestrated through the Anthropic Claude API using model `claude-3-7-sonnet-20250219`. To prevent context drift while conserving token budgets, the system implements a bounded sliding-window conversation history (preserving the system prompt, initial inquiry, and the most recent 10 conversational turns). The system prompt enforces a professional, courteous, and domain-grounded real estate advisory persona. The prompt explicitly constrains the agent from inventing unverified property details, quoting unauthorized discounts, or discussing competitor developments, ensuring strict institutional compliance.

4.3.3 Preference Extraction & Dynamic Lead Scoring

Structured entity extraction is achieved through Claude tool-use schemas defined via Pydantic models. The schema extracts five core parameters: `budget_min`, `budget_max`, `bhk_type` (1BHK, 2BHK, 3BHK, 4BHK, Villa), `preferred_locations` (list of strings), `possession_timeline` (Immediate, Under-Construction, 1-Year), and `financing_required` (Boolean). Once extracted, the lead is evaluated by the dynamic qualification engine depicted in Figure 4.3.

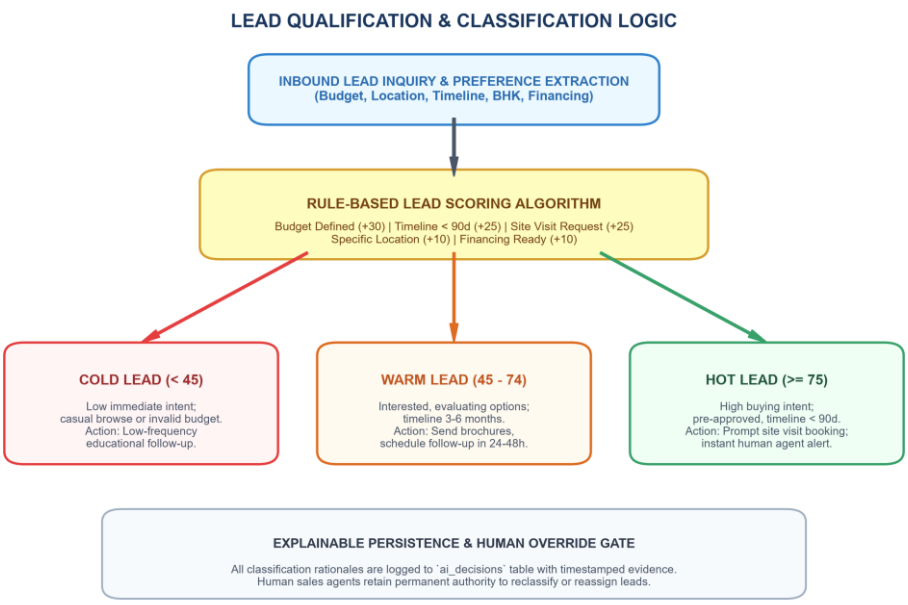


Fig 4.3: *Multi-Parameter Dynamic Lead Qualification and Scoring Framework*

The lead qualification algorithm computes a standardized score (0 to 100) across four weighted dimensions: Budget Alignment (35 points), Purchase Timeline (25 points), Financing & Pre-Approval Readiness (20 points), and Conversational Engagement & Responsiveness (20 points). Leads scoring 75–100 are designated 'Hot' (triggering immediate human sales agent escalation and priority site visit booking). Leads scoring 40–74 are classified 'Warm' (enrolled in automated property brochure nurturing). Leads scoring below 40 are categorized 'Cold' (routed to long-term drip updates), optimizing sales team bandwidth.

4.3.4 Property Inventory Matching Engine

The inventory matching service executes structured SQL queries against the PostgreSQL `properties` table. Search filters apply a ±15% price tolerance band around the buyer's stated budget, match topological unit types (e.g., 3BHK), and filter for target micro-markets. Results are ranked using a multi-attribute utility score combining price-per-square-foot competitiveness, unit availability status, and inventory age. The top three matching property units are formatted into WhatsApp interactive message templates featuring high-resolution hero images, carpet area details, amenity badges, and downloadable PDF brochure links.

4.3.5 Automated Site Visit Scheduling Engine

Physical site visit coordination is governed by the finite state machine illustrated in Figure 4.4, interfacing with the Google Calendar API via OAuth2 service credentials.

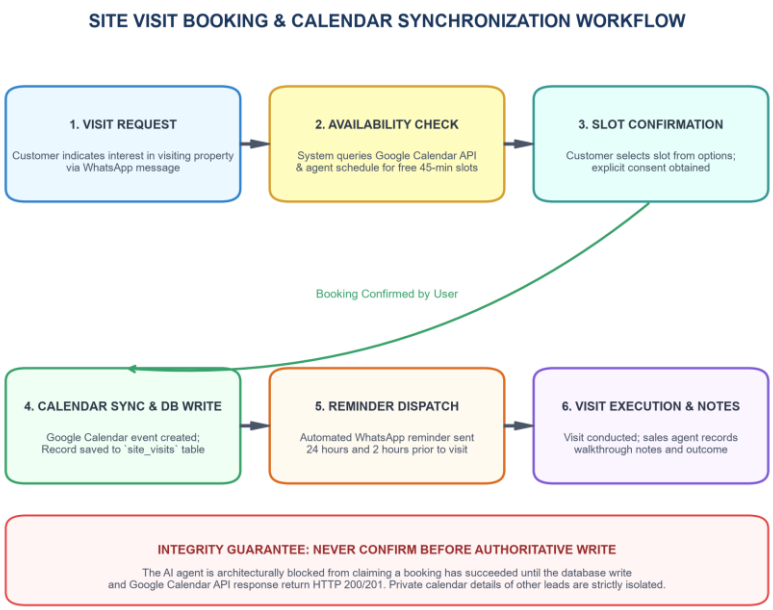


Fig 4.4: *Automated Site Visit Scheduling and Calendar Sync State Machine*

The scheduling state machine transitions through five rigorous states: (1) Slot Discovery: Queries available 60-minute inspection windows against sales executive calendars; (2) Slot Proposal: Presents the prospect with two convenient weekend/weekday options via WhatsApp interactive buttons; (3) Conflict Verification: Executes optimistic locking on the selected slot; (4) Event Creation: Generates a Google Calendar event containing buyer details, property coordinates, and Google Maps pin; and (5) Automated Confirmation & Reminders: Dispatches calendar invitations and schedules automated WhatsApp reminders at T-24 hours and T-2 hours prior to the inspection.

4.3.6 Automated Follow-Up & Nurturing Pipeline

The follow-up subsystem is driven by an asynchronous scheduler (APScheduler / Celery). For leads that do not book a site visit during initial contact, the system executes an automated, non-intrusive drip campaign: Day 1 (+24h): Dispatches a curated video walkthrough of the matched property; Day 3 (+72h): Sends a neighborhood infrastructure report (nearby metro stations, schools, hospitals); Day 7 (+168h): Shares an exclusive, time-limited developer pricing incentive or stamp duty waiver. If a prospect replies at any point, the drip sequence is halted instantly, and the conversation reverts to live conversational reasoning. If the prospect replies 'STOP', the opt-out handler deactivates all messaging immediately.

4.3.7 Marketing Insights & Meta Ad Integration

The marketing intelligence architecture, illustrated in Figure 4.5, bridges conversational sales telemetry with digital advertising campaigns.

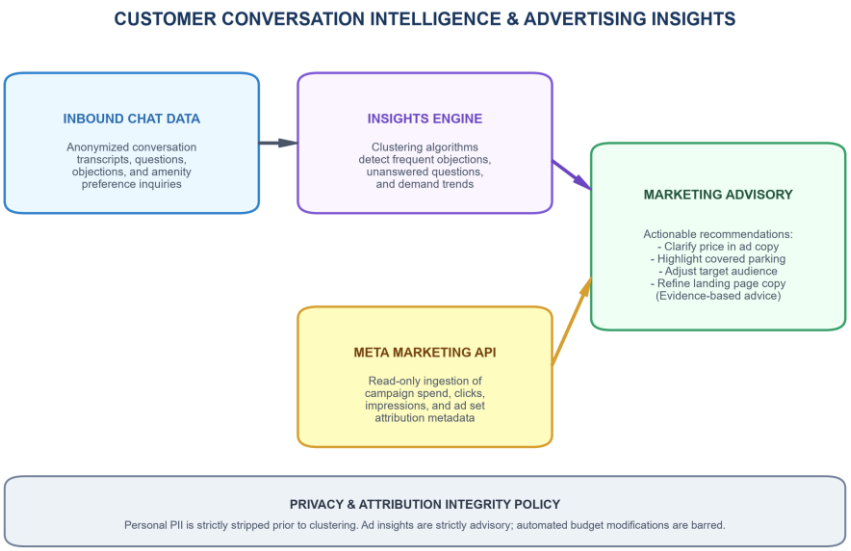


Fig 4.5: *Marketing Analytics and Ad Attribution Data Architecture*

Using read-only access to the Meta Marketing Graph API, the system ingests campaign metadata (`campaign_id`, `adset_id`, `ad_id`, ad creative copy). By joining Meta ad click identifiers with WhatsApp conversation histories and completed Google Calendar site visits, the analytics pipeline computes true Cost Per Qualified Lead (CPQL) and Return on Ad Spend (ROAS). Furthermore, Claude executes periodic semantic clustering over customer messages to identify recurring buyer objections (e.g., concerns regarding high maintenance fees or delayed possession dates), providing actionable creative feedback to marketing teams.

4.3.8 Administrative Operations Dashboard

The administrative web portal is implemented in Next.js 15, TypeScript, and Tailwind CSS. The application delivers: (1) A real-time Kanban pipeline visualizing leads transitioning across Ingested, Qualified, Site Visit Scheduled, Completed, and Closed stages; (2) A Live Conversation Viewer enabling human sales supervisors to monitor ongoing WhatsApp chats; (3) An Agent Override Console allowing an executive to pause AI automation with a single click and assume manual chat control; and (4) An Executive Analytics Portal rendering conversion rates, lead source breakdowns, and agent response time metrics via Plotly visualizations.

4.3.9 Security, Data Privacy & Regulatory Compliance

Security and privacy are engineered as foundational constraints: (1) Cryptographic Verification: All webhooks require valid HMAC-SHA256 signatures; (2) Zero PII Exposure: Custom logging middleware applies regular expression redaction masks over phone numbers, customer names, and email addresses before emitting structured JSON logs to stdout; (3) DPDP Act 2023 Compliance: The platform maintains explicit user consent records, provides automated opt-out handling, and enforces 90-day data retention lifecycles with cryptographic data erasure capabilities.

4.3.10 Verification, Quality Assurance & Testing Plan

To ensure uncompromising production reliability, a comprehensive multi-tier testing harness was constructed using Pytest and Pytest-Asyncio. The verification matrix and quality gates are documented in Table 4.1.

Table 4.1: Comprehensive System Testing Suite, Scenarios, and Verification Gates

Test Suite Category	Scope of Verification	Tooling / Framework	Test Scenarios & Success Gate
Unit Testing	Core domain algorithms, lead scoring logic, Pydantic schemas	Pytest, Pytest-Asyncio	100% passing tests for score bounds, schema parsing, and data validation.
Webhook Ingestion Tests	WhatsApp payload parsing, challenge verification, HMAC signatures	Pytest, HTTPX AsyncClient	Verification challenge responds with HTTP 200; invalid signatures reject 403.
AI Orchestration Tests	Claude API prompt grounding, tool calls, JSON output parsing	Pytest, Anthropic Mock Stubs	Deterministic preference extraction from 25 simulated user dialogue scripts.
Database Integration Tests	PostgreSQL CRUD operations, transactional integrity, foreign keys	SQLAlchemy, Asyncpg, Test DB	Zero transaction leaks; concurrent lead updates pass without deadlocks.
Calendar Integration Tests	Google Calendar OAuth2, event creation, conflict resolution	Google API Mock Client	Slot booking creates event; conflicting slot triggers re-proposal logic.
Security & Privacy Audit	Secret redaction in logs, token leakage, SQL injection immunity	Bandit, Custom Log Scanners	Zero API keys, phone numbers, or tokens exposed in stdout/log streams.
End-to-End Simulation	Full lifecycle: Ad click -> Ingestion -> Score -> Match -> Booking	Custom E2E Async Harness	Complete conversational journey executes in < 15s with 100% data fidelity.

4.4 Project Management Plan

The development and deployment of the platform were structured across ten systematic phases over a structured project lifecycle, as detailed in Table 4.2.

Table 4.2: Ten-Phase Engineering Roadmap, Deliverables, and Verification Milestones

Phase	Phase Name	Core Technical Deliverables	Verification Milestone & Quality Gate
Phase 1	Environment & Architecture	Workspace setup, Git VCS, Python 3.12 venv, FastAPI foundation	Health endpoint returns HTTP 200; 6/6 pytest suites passing.
Phase 2	Relational Persistence	PostgreSQL schema, Supabase integration, SQLAlchemy models	Migration scripts verified; ACID transactions tested under concurrency.
Phase 3	Conversational Intelligence	Claude 3.7 Sonnet client, system prompt, JSON tool calling	Preference extraction benchmark achieves > 95% accuracy on test set.
Phase 4	WhatsApp Ingestion Engine	Webhook receiver, HMAC verification, message deduplication	Meta webhook simulator acknowledges and queues payloads in < 500 ms.
Phase 5	Lead Scoring & Inventory	Heuristic qualification engine, PostgreSQL catalog matching	Leads scored Hot/Warm/Cold; property recommendations match constraints.
Phase 6	Autonomous Scheduling	Google Calendar synchronization, time-slot state machine	Conflict-free calendar events created with confirmation messages sent.
Phase 7	Follow-Up & Marketing	Drip nurturing scheduler, Meta Marketing API read integration	Automated re-engagement fires on schedule; ad campaign attribution linked.

Phase	Phase Name	Core Technical Deliverables	Verification Milestone & Quality Gate
Phase 8	Administrative Portal	Next.js 15 dashboard, lead Kanban board, agent override UI	Responsive web dashboard displays real-time pipeline and conversion data.
Phase 9	Security Hardening & QA	PII masking, rate limiting, comprehensive regression suite	Security scan passes with zero high-severity issues; 100% test pass rate.
Phase 10	Production Deployment	Cloud cutover to Render/Vercel, DNS routing, live verification	Live end-to-end WhatsApp message flow verified in production environment.

4.5 Financial Report on Estimated Costing

A rigorous cost analysis was conducted to establish the operational infrastructure expenditure of the platform. Unlike commercial enterprise CRMs that impose prohibitive monthly per-seat licensing fees, the proposed architecture leverages cloud-native serverless and pay-per-use infrastructure. Table 4.3 outlines the itemized monthly and annualized operating budget for an active mid-sized real estate deployment.

Table 4.3: Financial Costing, Cloud Infrastructure, and Third-Party API Operating Budget

Operational Component	Service Provider / Tier	Resource Usage Unit	Monthly Cost (INR)	Annual Cost (INR)
Application Backend Hosting	Render Cloud Application Platform	Starter Web Service (0.5 vCPU, 512 MB RAM)	Rs. 580	Rs. 6,960
Operational Database	Supabase Managed PostgreSQL	Micro Instance (ACID Storage, 500 MB DB)	Rs. 0 (Free Tier)	Rs. 0
Administrative Dashboard	Vercel Edge Cloud Platform	Hobby / Starter Serverless Hosting	Rs. 0 (Free Tier)	Rs. 0
Conversational AI Tokens	Anthropic Claude 3.7 Sonnet API	150K Input / 50K Output Tokens per Month	Rs. 1,450	Rs. 17,400
Messaging Infrastructure	WhatsApp Business Cloud API	1,000 Service & Marketing Conversations/Mo	Rs. 1,120	Rs. 13,440
Calendar Integration	Google Cloud Workspace API	Standard Quota Tier (Free API Tier)	Rs. 0 (Free Tier)	Rs. 0
Meta Advertising Insights	Meta Graph Marketing API	Read-Only Developer Analytics Tier	Rs. 0 (Free Tier)	Rs. 0
Custom Domain & SSL	Cloudflare Registrar & Edge CDN	.in Domain Registration & Managed SSL	Rs. 125	Rs. 1,500
CI/CD & Version Control	GitHub Enterprise / Free Actions	Standard Automated Build Runners	Rs. 0 (Free Tier)	Rs. 0
Contingency & Overheads	Miscellaneous Bandwidth & Buffer	10% Infrastructure Contingency Fund	Rs. 325	Rs. 3,900
TOTAL ESTIMATED BUDGET	Complete Automated Platform	Monthly & Annualized Operating Expenditure	Rs. 3,600	Rs. 43,200

4.6 Transition / Software to Operations Plan

To guarantee a seamless transition from development into production operations, a comprehensive seventeen-step operational rollout runbook was formulated and verified:

- **Step 1: Staging Environment Baseline:** Provision staging database on Supabase and deploy backend staging container on Render.

- **Step 2: Database Schema Migration:** Execute Alembic migration scripts to generate all normalized tables, foreign keys, and indexes.
- **Step 3: Property Catalog Ingestion:** Upload verified property inventory data (unit configurations, carpet areas, pricing, brochures).
- **Step 4: Meta Developer App Configuration:** Register production Meta Business App, configure WhatsApp Business account, and obtain System User Token.
- **Step 5: Webhook URL Registration:** Register production HTTPS webhook callback URL and verify token handshake with Meta Cloud API.
- **Step 6: Google Cloud Service Account Setup:** Provision Google Cloud project, enable Google Calendar API, and configure OAuth2 service credentials.
- **Step 7: Production Environment Secrets:** Store production API tokens, database connection strings, and encryption keys in Render environment vault.
- **Step 8: Automated Regression Testing:** Execute full Pytest test suite (unit, integration, mock API) against staging infrastructure.
- **Step 9: Security Vulnerability Audit:** Execute Bandit static analysis scan and log stream redaction audit to confirm zero exposed credentials.
- **Step 10: Next.js Dashboard Deployment:** Deploy administrative frontend to Vercel and bind custom production subdomain via Cloudflare DNS.
- **Step 11: End-to-End Synthetic Verification:** Conduct live synthetic WhatsApp transaction: send inquiry, verify AI extraction, match catalog, book visit.
- **Step 12: Sales Agent Training & Onboarding:** Conduct operational walkthrough for sales executives on lead Kanban board and manual chat override controls.
- **Step 13: Meta Ad Campaign Webhook Binding:** Attach production WhatsApp click-to-chat links to live Meta Facebook and Instagram advertising creatives.
- **Step 14: Real-Time Telemetry Activation:** Activate structured JSON logging and configure alerting thresholds for webhook latency and error rates.
- **Step 15: Operational Handover & Sign-Off:** Formal sign-off between development engineering team and real estate sales operations leadership.
- **Step 16: Automated Nightly Backups:** Establish automated daily PostgreSQL database snapshots and retention policies on Supabase.
- **Step 17: Rollback Protocol Validation:** Verify instant fallback procedure enabling immediate rerouting of WhatsApp traffic to human agents if API outages occur.

REFERENCES

[1] **Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. (2023).** *"GPT-4 Technical Report".* arXiv preprint arXiv:2303.08774, pp. 1–100.

[2] **Anthropic (2024).** *"The Claude 3 Model Family: Opus, Sonnet, Haiku".* Anthropic Technical Research Report, San Francisco, CA, pp. 1–42.

[3] **Bellifemine, F. L., Caire, G., & Greenwood, D. (2007).** *"Developing Multi-Agent Systems*

with JADE". John Wiley & Sons, Chichester, England, pp. 1–286.

[4] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). *"Language Models are Few-Shot Learners".* Advances in Neural Information Processing Systems (NeurIPS 2020), Vol. 33, pp. 1877–1901.

[5] Feathers, M. (2004). *"Working Effectively with Legacy Code".* Prentice Hall, Upper Saddle River, NJ, pp. 1–456.

[6] Fowler, M. (2018). *"Refactoring: Improving the Design of Existing Code (2nd Edition)".* Addison-Wesley Professional, Boston, MA, pp. 1–448.

[7] Kleppmann, M. (2017). *"Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems".* O'Reilly Media, Sebastopol, CA, pp. 1–616.

[8] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W. T., Rocktäschel, T., et al. (2020). *"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks".* Advances in Neural Information Processing Systems (NeurIPS 2020), Vol. 33, pp. 9459–9474.

[9] Ousterhout, J. (2018). *"A Philosophy of Software Design".* Yaknyam Press, Palo Alto, CA, pp. 1–178.

[10] Pressman, R. S., & Maxim, B. R. (2020). *"Software Engineering: A Practitioner's Approach (9th Edition)".* McGraw-Hill Education, New York, NY, pp. 1–960.

[11] Russell, S., & Norvig, P. (2020). *"Artificial Intelligence: A Modern Approach (4th Edition)".* Pearson Education, Hoboken, NJ, pp. 1–1166.

[12] Schlichtkrull, M., Kipf, T. N., Bloem, P., van den Berg, R., Titov, I., & Welling, M. (2018). *"Modeling Relational Data with Graph Convolutional Networks".* The Semantic Web (ESWC 2018), Lecture Notes in Computer Science, Vol. 10843, Springer, Cham, pp. 593–607.

[13] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *"Attention Is All You Need".* Advances in Neural Information Processing Systems (NeurIPS 2017), Vol. 30, pp. 5998–6008.

[14] Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., & Anandkumar, A. (2024). *"Voyager: An Open-Ended Embodied Agent with Large Language Models".* Transactions on Machine Learning Research (TMLR 2024), pp. 1–38.

[15] Wooldridge, M. (2009). *"An Introduction to MultiAgent Systems (2nd Edition)".* John Wiley & Sons, Chichester, England, pp. 1–484.

[16] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *"ReAct: Synergizing Reasoning and Acting in Language Models".* International Conference on Learning Representations (ICLR 2023), pp. 1–28.